Android Architecture

1. Introduction

- Android is a Linux-based mobile operating system developed by Google.
- It follows a layered architecture that separates concerns and provides a flexible, secure and scalable platform for app development.
- The Android architecture consists of several layers, each with a specific role, from hardware to the application layer.

2. Layers in Android Architecture

1. Linux Kernel (Bottom Layer)

- Provides core system services like process management, memory management, device drivers, security, and power management.
- Handles communication between hardware and software.

2. Hardware Abstraction Layer (HAL)

- Acts as a bridge between the Linux kernel and higher layers.
- Provides a standard interface for device-specific features like camera, GPS, audio, etc.

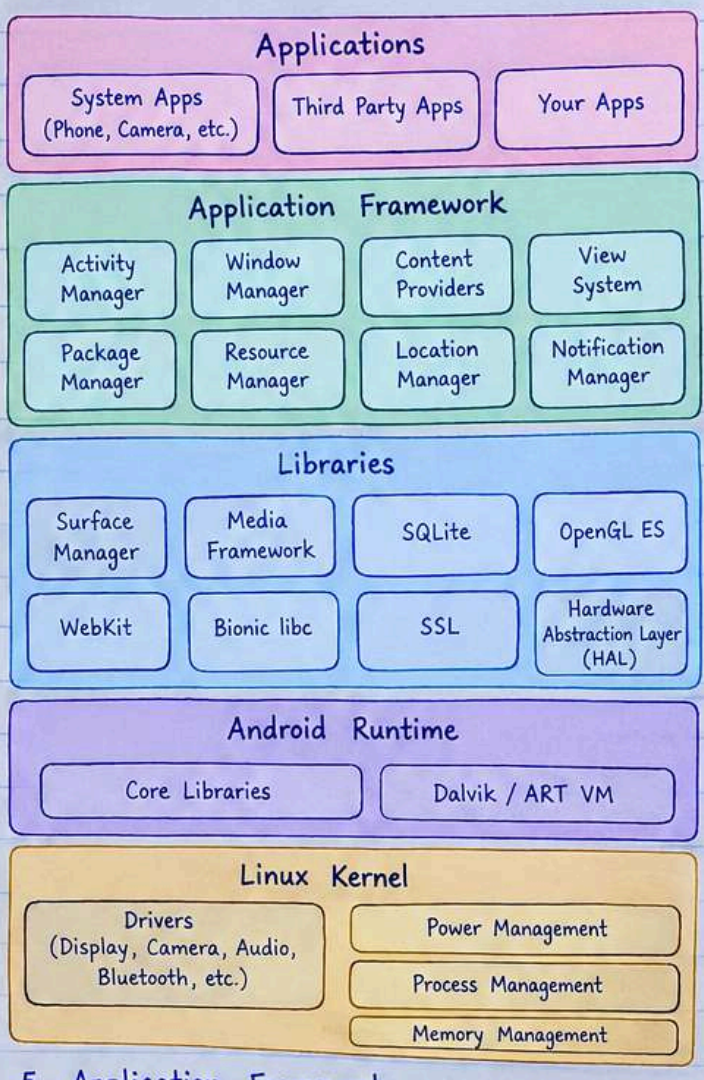
3. Native Libraries

- Includes C/C++ libraries that provide core functionalities.
- Examples: Surface Manager (display), Media Framework (audio/video), SQLite (database), OpenGL ES (graphics), WebKit (web browser), Bionic libc (C library), SSL (security).

4. Android Runtime

- Provides the runtime environment for Android apps.
- Uses ART (Android Runtime) from Android 5.0 onwards (replaces Dalvik VM).
- Includes core libraries and manages memory, garbage collection, and app execution.

Android Architecture Diagram



5. Application Framework

- Provides high-level services to applications.
- Includes Activity Manager, Window Manager, Content Providers, View System, Package Manager, Resource Manager, Location Manager, Notification Manager, etc.

6. Applications (Top Layer)

- Contains pre-installed system apps (Phone, Camera, Settings, etc.) and third-party apps.
- Developers build their own apps using the Android SDK, which interacts with the framework layer.

3. Key Points

- The architecture is layered for modularity, security and ease of development.
- Each layer provides services to the layer above and uses services from the layer below.
- Android's architecture allows device manufacturers to customize hardware while keeping the same application framework for developers.

Kotlin

1. Kotlin & Java

- Kotlin is a modern, statically typed programming language developed by JetBrains.
- It is widely used for Android application development.

Kotlin vs Java

Kotlin	Java
Modern language	Older language
Developed by JetBrains	Developed by Sun Microsystems
Concise syntax	More verbose syntax
Null safety	NullPointerException common
Extension functions	Not directly supported
Smart casts	Requires explicit casting
Semicolon optional	Semicolon normally required
Coroutines	Uses threads/executors
Fully interoperable	Can work with Kotlin code

Example:

Java:
```
String name = "Ashwini";
System.out.println(name);
```

Kotlin:
```
val name = "Ashwini"
println(name)
```

→ Kotlin requires less code while providing similar functionality.

2. Java to Kotlin

- Kotlin is designed to work with existing Java programs. A Java project can be gradually converted into Kotlin.

Example:

Java:
```
public class Student{
    String name = "Ravi";
    void display(){
        System.out.println(name);
    }
}
```

Kotlin:
```
class Student{
    val name = "Ravi"
    fun display(){
        println(name)
    }
}
```

Important differences:

- Java: `int age = 20;`
  Kotlin: `var age: Int = 20`
- Java:
```
if (name != null) {
    System.out.println(name);
}
```
Kotlin:
```
println(name?.length)
```

Java → Kotlin conversion in Android Studio:

1. Open the Java file.
2. Select the Java code.
3. Choose Code → Convert Java File to Kotlin File.
4. Check and modify the generated code if required.

Advantages:

- Reduces code size
- Provides null-safety
- Supports modern features
- Easier Android development
- Java and Kotlin can work together.

6. Kotlin Variables

- A variable is a named memory location used to store a value.
- Types: val, var

val (read-only reference)
- cannot be reassigned
- Example: `val age = 20`
- `age = 25` (error)

var (mutable reference)
- can be reassigned
- Example: `var age = 20`
- `age = 25` (valid)

3. Kotlin & Android Studio

Android Studio is the official IDE used for Android development. Kotlin has strong support in Android Studio.

Features of Kotlin in Android Studio:

1. Code completion – suggests keywords, methods, variables.
2. Syntax highlighting – makes code easier to read.
3. Error detection – identifies errors while writing code.
4. Java-to-Kotlin conversion – converts Java source code.
5. Debugging – helps find and fix errors.
6. Refactoring – allows safe code modification.
7. Gradle integration – manages dependencies and builds.
8. Kotlin compiler support – compiles Kotlin code.
9. Android SDK integration – provides access to Android APIs.
10. Emulator support – test apps on virtual devices.

Creating a Kotlin Android project:

1. Android Studio → New Project
2. Select template → Configure project
3. Select Kotlin → Finish

Project structure (example):

```
app/
  manifests/
  java/
    MainActivity.kt
  res/
    drawable/
    layout/
    values/
  Gradle Scripts
```

4. Semicolon in Kotlin

- In Kotlin, semicolon (;) is optional at the end of a statement.
- Example:
```
val name = "Ravi"
val age = 20
println(name)
```
Semicolon can still be used:
```
val name = "Ravi";
val age = 20;
```
- Multiple statements on one line:
```
val x = 10; val y = 20
```
- Kotlin uses line breaks to separate statements.
- Semicolons are optional, unlike Java.

5. Kotlin Data Types

- A data type specifies what kind of value a variable can store.
- Main types: Numbers, Character, Boolean, String, Array, Nullable

A. Number types:

Type	Size	Example
Byte	8 bits	`val b: Byte = 25`
Short	16 bits	`val s: Short = 1000`
Int	32 bits	`val i: Int = 20`
Long	64 bits	`val l: Long = 1400000000L`
Float	32 bits	`val f: Float = 99.5F`
Double	64 bits	`val d: Double = 3.14159`

B. Others:

- Char – single character ('A')
- Boolean – true/false (`val b: Boolean = true`)
- String – sequence of characters ("Hello")
- Array – collection (`arrayOf(1,2,3)`)
- Nullable – can hold null (`String?`)

7. Variable Declaration

- Syntax:
```
val variableName: DataType = value
```
or
```
var variableName: DataType = value
```
- Examples:
```
val name: String = "Ashwini"
var age: Int = 20
val salary: Double = 25000.50
var grade: Char = 'A'
val passed: Boolean = true
```

Type inference:
```
val name = "Ashwini"   → String
val age = 20           → Int
val price = 99.5       → Double
```
(Type inferred by compiler)

Difference between val and var:

val	var
Read-only reference	Mutable reference
Cannot be reassigned	Can be reassigned
Example: `val age = 20`	Example: `var age = 20`

Type Annotation & Inference, Nullable, Safe Call Operator, Not Null Assertion, Nullable Type and let Function, Elvis Operator, Type Casting & Checking

1. Type Annotation & Inference

(a) Type Annotation :

- In Kotlin, we can explicitly specify the data type of a variable using a type annotation.
- It is written using a colon (:) followed by the data type.

Syntax : `val variableName: DataType = value`

Example :

```
val name: String = "Ashwini"
var age: Int = 20
```

(b) Type Inference :

- Kotlin can automatically determine the type of a variable from the assigned value.
- This is called type inference.

Example :

```
val name = "Ashwini"   // inferred as String
val age = 20           // inferred as Int
val price = 99.5       // inferred as Double
```

2. Nullable

- By default, Kotlin does not allow null values for variables (non-nullable).
- To allow null, we use a question mark (?) after the type.

Example :

```
var name: String? = null
println(name)
```

- Without ?, the following gives an error:

```
var name: String = null    // Error
```

3. Safe Call Operator (?.)

- The safe call operator (?.) is used to access a member or call a function on a nullable type.
- If the value is null, it returns null instead of throwing a NullPointerException.

Example :

```
val name: String? = "Ashwini"
println(name?.length)    // returns length or null
```

Key Points to Remember :

- Use type annotation when the type is not obvious.
- Use type inference for shorter code.
- Use nullable types (?) when null is possible.
- Use safe call (?.) and let {} for safe operations.
- Use ?: for default values.
- Use !! only when you are sure the value is not null.
- Use as / as? for type casting.
- Use is for type checking.

4. Not Null Assertion (!!)

- The not null assertion operator (!!) tells the compiler that the value will never be null.
- It is used when we are sure that the value is not null, but the compiler cannot know it.

Example :

```
val name: String? = "Ashwini"
println(name!!.length)   // length is 7
```

* If the value is actually null, it throws a NullPointerException.

5. Nullable Type and let Function

(a) Nullable Type :

- A nullable type is a type that can hold null.
- It is written using a question mark (?).

Example :

```
var address: String? = null
```

(b) let Function :

- The let function is used to perform an operation only if the value is not null.
- It avoids explicit null checks.

Example :

```
val name: String? = "Ashwini"
val length = name?.let { it.length }
println(length)   // 7
```

6. Elvis Operator (?.)

- The Elvis operator (?:) provides a default value if the value is null.
- It is a shorthand way of writing a null check.

Example :

```
val name: String? = null
val result = name ?: "Unknown"
println(result)    // Unknown
```

7. Type Casting & Checking

(a) Type Casting :

- Used to convert a variable from one type to another.
- In Kotlin, we use as (safe cast) or as? (safe cast that returns null if cast fails).

Example :

```
val obj: Any = "Hello"
val str = obj as String    // Normal cast
```

(b) Type Checking :

- Used to check whether a variable is of a particullar type.
- It is done using the is operator.

Example :

```
val obj: Any = "Hello"
if (obj is String) {
    println("It is a String")
}
```

Operators & Expression :

- Expression, Assignment, Arithmetic, Augmented Assignment, Increment & Decrement, Equality, Boolean Logical, Range & Bitwise.

Flow Control (Loops & condition).

1. Expression

- An expression is a combination of values, variables and operators that produces a value.
- In Kotlin, expressions can be used wherever a value is expected.

Example:

```
val result = 10 + 5
println(result)   // 15
```

2. Assignment Operator

- Used to assign a value to a variable.
- The most common operator is =.

Example:

```
var name = "Kotlin"
name = "Android"
```

3. Arithmetic Operators

- Used to perform mathematical operations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

Example:

```
val a = 10
val b = 3
println(a + b)   // 13
println(a % b)   // 1
```

4. Augmented Assignment Operators

- Combines an arithmetic operation with assignment.

Operators: +=, -=, *=, /=, %=

Example:

```
var a = 10
a += 5     // a = a + 5   (15)
a -= 3     // a = a - 3   (12)
a *= 2     // a = a * 2   (24)
a /= 4     // a = a / 4   (6)
```

5. Increment & Decrement Operators

- Used to increase or decrease the value by 1.

Operators: ++ (increment), -- (decrement)

Forms: ++a (pre-increment), a++ (post-increment)
--a (pre-decrement), a-- (post-decrement)

Example:

```
var count = 5
println(++count)   // 6  (pre-increment)
println(count++)   // 6  (post-increment)
```

6. Equality Operators

- Used to compare two values.

Operators: == (equal to), != (not equal to)

Example:

```
val a = 5
val b = 10
println(a == b)   // false
println(a != b)   // true
```

7. Boolean Logical Operators

- Used to perform logical operations on boolean values.

Operators: && (AND), || (OR), ! (NOT)

Example:

```
val a = true
val b = false
println(a && b)   // false
println(a || b)   // true
println(!a)       // false
```

8. Range Operator

- Used to create a range of values.

Operator: .. (two dots)

Example:

```
for (i in 1...5) {
    println(i)
}          // 1 2 3 4 5
```

9. Bitwise Operators

- Used to perform bit-level operations on integers.

Operators: and (&), or (|), xor (^), not (^), shl (<<), shr (>>)

Example:

```
val a = 5     // 0101
val b = 3     // 0011
println(a and b)   // 1   (0001)
println(a or b)    // 7   (0111)
println(a xor b)   // 6   (0110)
println(a shl 1)   // 10  (1010)
println(a shr 1)   // 2   (0010)
```

10. Flow Control (Loops & Condition)

A. Conditional Statements

1. if statement

- Executes a block of code if the condition is true.

Example:

```
val age = 20
if (age >= 18) {
    println("Adult")
} else {
    println("Minor")
}
```

2. when statement

- Similar to switch case.

Example:

```
val day = 1
when (day) {
    1 -> println("Mon")
    2 -> println("Tue")
    else -> println("Other")
}
```

B. Loops

1. for loop

- Used to iterate over a range or collection.

Example:

```
for (i in 1..5) {
    println(i)
}
```

2. while loop

- Executes a block while the condition is true.

Example:

```
var i = 1
while (i <= 5) {
    println(i)
    i++
}
```

3. do-while loop

- Executes the block at least once.

Example:

```
var i = 1
do {
    println(i)
    i++
} while (i <= 5)
```

Functions & Lambda

1. Type

- In Kotlin, functions can have a type, which specifies the type of value they return.
- The type is written after the parameter list using the ":" symbol.
- If a function does not return any value, the return type is Unit.

Example:

```
fun add(a: Int, b: Int): Int {
    return a + b
}
fun printMessage(msg: String): Unit {
    println(msg)
}
```

Here:
- add() returns Int
- printMessage() returns Unit (no value)

2. Return Values

- A function can return a value using the return keyword.
- The return value is of the type specified in the function declaration.
- If there is no return value, the function is declared with Unit.

Example:

```
fun getMax(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

This function returns the greater number as an Int.

3. Parameters

- Parameters are the input values passed to a function.
- They are defined inside the function parentheses.
- We can have multiple parameters separated by commas.

Example:

```
fun multiply(a: Int, b: Int): Int {
    return a * b
}
```

Here, a and b are parameters of type Int.

4. Variable Number of Arguments

- Kotlin provides vararg to accept a variable number of arguments.
- It can be used only for the last parameter.
- The arguments are treated as an array inside the function.

Example:

```
fun sum(vararg numbers: Int): Int {
    var total = 0
    for (n in numbers) {
        total += n
    }
    return total
}
```

Here, numbers is a variable number of arguments.

sum(1,2,3,4) → 10

5. Lambda (Anonymous) Functions

- Lambda is a short form of a function.
- It is an anonymous function (without a name).
- It is commonly used with higher order functions.

Syntax:

```
{ parameters → expression }
```

Example:

```
val add: (Int, Int) → Int = { a, b → a + b }

val greet: (String) → Unit = { name →
        println("Hello, $name")
}
```

Here, add is a lambda that takes two Int parameters and returns an Int.

It takes a String parameter and returns Unit.

6. Higher Order Functions

- A higher order function is a function that takes another function as a parameter or returns a function.
- Kotlin provides many higher order functions like map(), filter(), reduce(), sorted(), etc.

Example:

```
val numbers = listOf(1,2,3,4,5)
val doubled = numbers.map { it * 2 }
println(doubled)    // [2, 4, 6, 8, 10]
```

map() takes a lambda function as an argument and returns a new list.

Basics of OOP's (Object Oriented Programming)

1. What is OOP?

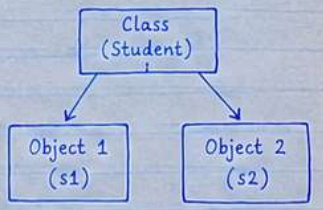
- OOP is a programming paradigm based on the concept of objects.
- It helps to organize code in a structured way and makes it reusable and easier to maintain.
- Four main concepts: Class, Object, Encapsulation, Inheritance, Polymorphism, Abstraction.

2. Class and Object

- Class is a blueprint or template for creating objects.
- Object is an instance of a class.

Example:

```
class Student {
    var name: String = ""
    var age: Int = 0
}
val s1 = Student()  // Creates an object
s1.name = "Riya"
s1.age = 20
```



3. Inheritance

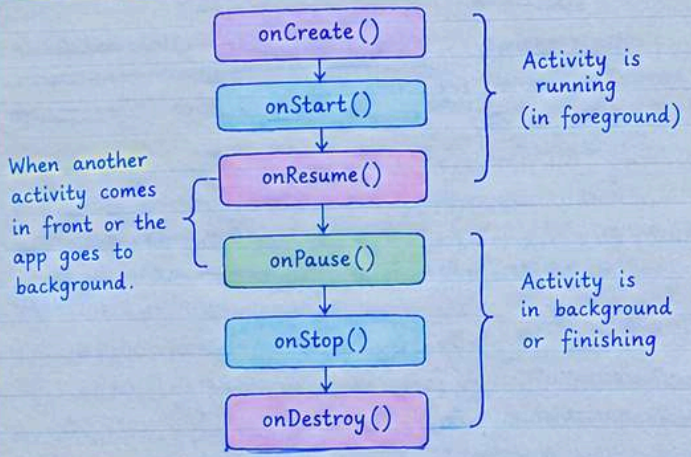
- Inheritance allows a class to acquire properties and methods from another class.
- It helps in code reuse and reduces duplication.
- In Android, we use inheritance to extend existing classes such as Activity, Fragment, etc.

Activity Life Cycles, Activity State Changes, Save & Restore State - Android

1. Activity Life Cycle

- An Activity represents a single screen with a user interface in Android.
- The activity life cycle defines the different states an activity goes through from creation to destruction.
- The system calls specific callback methods of an activity at each stage.

Activity Life Cycle Methods



Description of Important Methods

Method	Description
onCreate()	Initializes the activity. Set the layout and initialize variables.
onStart()	Activity becomes visible to the user.
onResume()	Activity is in the foreground and ready for interaction.
onPause()	Another activity is taking focus. Save lightweight data.
onStop()	Activity is no longer visible.
onDestroy()	Activity is being destroyed. Clean up resources.

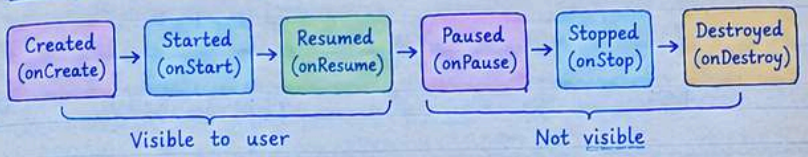
★ **Remember:**

- onCreate() is called only once.
- onStart() and onResume() can be called multiple times.
- onDestroy() is called before the activity is destroyed.

2. Activity State Changes

- State changes occur when the user interacts with the app or when the system needs to manage resources.
- The system moves the activity between different states based on events like:
  - User pressing Home button
  - Incoming call
  - Switching to another app
  - Screen rotation (configuration change)

Example: State Changes



3. Save & Restore State

- Sometimes the system destroys an activity (e.g., due to low memory or screen rotation).
- To preserve the data, we save the state in onSaveInstanceState() and restore it in onCreate() or onRestoreInstanceState().

1. onSaveInstanceState()

- Called before the activity is destroyed.
- Used to save data in a Bundle object.

Example:

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putString("name", userName);
    outState.putInt("count", count);
}
```

Here, we save the values of userName and count.

2. onCreate() - Restore State

- Called when the activity is created.
- We can get the saved data from the Bundle object.

Example:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    if (savedInstanceState != null) {
        userName = savedInstanceState.getString("name");
        count = savedInstanceState.getInt("count");
    }
}
```

Here, we restore the values if savedInstanceState is not null.

★ **Key Points:**

- Use onSaveInstanceState() to save small amounts of data.
- Use onCreate() or onRestoreInstanceState() to restore the data.
- For large data, use a database or SharedPreferences.

1. Views in Android

1. Definition

- A View is the basic building block of the Android user interface. It represents a rectangular area on the screen that can display information or receive user interaction.

2. Common Views

View	Description
TextView	Displays text
Button	Performs an action
EditText	Accepts user input
ImageView	Displays an image
CheckBox	Allows multiple selection
RadioButton	Allows single selection

3. Example

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android" />
```

Here, TextView is a View.

1. View Groups

1. Definition

- A ViewGroup is a special type of View that acts as a container for other Views and ViewGroups.
- It does not have a visual appearance by itself, but it arranges and manages the child views.

2. Common ViewGroups

- LinearLayout → arranges views in a row or column
- FrameLayout → places views on top of each other
- RelativeLayout → positions views relative to each other
- ConstraintLayout → uses constraints for positioning

3. Example

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView ... />
    <EditText ... />
    <Button ... />
</LinearLayout>
```

Here, LinearLayout is a ViewGroup and it contains TextView, EditText and Button.

Layouts in Android

1. Definition

- A Layout defines how the UI components are arranged and positioned on the screen.
- It controls the position, size, alignment, spacing and orientation of the views.

2. Types of Layouts

a) LinearLayout

- Arranges components in a row or column.

Example (vertical):

```
<LinearLayout
    android:orientation="vertical">
    <TextView ... />
    <Button ... />
</LinearLayout>
```

Example (horizontal):

```
<LinearLayout
    android:orientation="horizontal">
    <Button ... />
    <Button ... />
</LinearLayout>
```

b) ConstraintLayout

- Uses constraints to position views.
- Useful for complex and responsive UI.

Example:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView ... />
    <Button ... />
</androidx.constraintlayout.widget.ConstraintLayout>
```

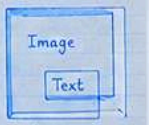
Here, Button can be constrained below the TextView.

c) FrameLayout

- Places views on top of each other.

Example:

```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView ... />
    <TextView ... />
</FrameLayout>
```



(on top)

d) ScrollView

- Used when content is larger than the available screen.

Example:

```
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout>
        <!-- large content -->
    </LinearLayout>
</ScrollView>
```

Allows vertical scrolling.

3. Difference Between View, ViewGroup and Layout

Concept	Meaning	Examples
View	Individual UI element	Button, TextView
ViewGroup	Container for Views	LinearLayout
Layout	Arranges UI elements	LinearLayout, ConstraintLayout

4. Key Points

- View = single UI element
- ViewGroup = container
- Layout = arrangement of views

Services in Android

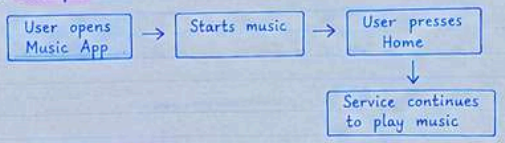
1. Definition

- A Service is an Android application component designed to perform work that can continue without a user interface.
- It runs in the background and can be started, stopped or destroyed by the system.

2. Common Uses

- Playing music
- Performing ongoing operations
- Monitoring certain events
- Processing work in the background

3. Example



4. Important Note

- A Service does not normally provide a UI.
- For long running user-visible work, Android uses Foreground Services. For deferrable background work, WorkManager is recommended.

5. Activity vs Service

Activity	Service
Provides UI	Normally no UI
User interacts with it	Performs background work
Represents a screen	Represents ongoing work

Intent in Android

1. Definition

- An Intent is a messaging object used to request an action from another Android component.
- It can be used to:
  - Start another Activity
  - Start a Service
  - Send a broadcast
  - Pass data between components.

2. Types of Intent

a) Explicit Intent

- Specifies the exact component to be started.
- Used to move from one Activity to another.

Example:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

b) Implicit Intent

- Does not specify a particular component.
- Describes the action to be performed.

Example:

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = Uri.parse("https://www.google.com")
startActivity(intent)
```

3. Demonstration: Intent to Start Another Activity

Steps:

1. Create first Activity layout (activity_main.xml)
2. Write MainActivity.kt with Intent.
3. Create SecondActivity layout and class.
4. Declare second activity in AndroidManifest.xml.
5. Run the application.

